



T.C.

BARTIN ÜNİVERSİTESİ

BARTIN MESLEK YÜKSEKOKULU

BİLGİSAYAR TEKNOLOJİLERİ BÖLÜMÜ

BİLGİSAYAR PROGRAMCILIĞI PROGRAMI

ALGORİTMA ÖDEVİ RAPORU

JavaScript ile Masaüstü Uygulama Geliştirme:

Electron.js ve Tarif Analizi

NİSA NUR AKDEMİR

25015221002

Danışman:

Dr. Öğr.Serkan Aksu

BARTIN- 2025-2026 Bahar

İçindekiler

1. GİRİŞ: JAVASCRIPT VE MASAÜSTÜ UYGULAMA GELİŞTİRME	3
2. KULLANILAN BAŞLICA FRAMEWORK, KÜTÜPHANE VE ARAÇLAR	4
2.1 Electron.js	4
2.1.1 Chromium Motoru	4
2.1.2 Node.js	5
2.2 Tauri	6
2.3 Diğer Yardımcı Araçlar ve Kütüphaneler	7
3. ALANIN AVANTAJLARI VE SINIRLILIKLARI	8
3.1. Avantajlar	8
3.2. Sınırlılıklar ve Teknik Zorluklar	8
4. GERÇEK DÜNYADAN UYGULAMA ÖRNEKLER	9
4.1 Visual Studio Code (VS Code)	9
4.1.1 VS Code'un Avantajları	10
4.2. Discord	11
5. BASİT BİR KOD PARÇASI (SNIPPET) İLE ÖRNEK GÖSTERİM	12
6. SONUÇ VE DEĞERLENDİRME	13
Kaynakça	14

1. GİRİŞ: JAVASCRIPT VE MASAÜSTÜ UYGULAMA GELİŞTİRME

Günümüzde yazılım dünyasının en dinamik dillerinden biri olan JavaScript, başlangıçta yalnızca web tarayıcıları içerisinde etkileşimli içerikler sunmak amacıyla geliştirilmiştir. Ancak zaman içerisinde geçirdiği evrim ve geliştirilen Node.js gibi çalışma ortamları sayesinde, tarayıcı sınırlarını aşarak sunucu tarafı geliştirmeden nesnelerin internetine kadar pek çok farklı alanda kullanılmaya başlanmıştır. Bu alanlar arasında en dikkat çekici olanlardan biri de "Masaüstü Uygulama Geliştirme" sürecidir.

Geleneksel masaüstü uygulama geliştirme süreçlerinde C++, C# veya Java gibi dillerin kullanımı yaygın olsa da, modern yazılım ekosisteminde web teknolojilerinin (HTML, CSS ve JavaScript) gücü masaüstü platformlarına taşınmıştır. Bu yaklaşım, geliştiricilere tek bir kod tabanı üzerinden Windows, macOS ve Linux gibi farklı işletim sistemlerine uyumlu uygulamalar üretme imkânı tanımaktadır. Özellikle çapraz platform desteği, geliştirme maliyetlerini düşürmesi ve zengin kullanıcı arayüzü (UI/UX) olanakları sunması nedeniyle hem bireysel geliştiriciler hem de büyük teknoloji şirketleri tarafından öncelikli olarak tercih edilmektedir.

Bu rapor kapsamında, JavaScript kullanarak masaüstü uygulaması geliştirmenin teknik altyapısı incelenecek; Electron.js ve Tauri gibi güncel araçların çalışma prensipleri, avantajları ve sınırlılıkları karşılaştırmalı olarak ele alınacaktır. Ayrıca, günümüzde yaygın olarak kullanılan profesyonel yazılım örnekleri üzerinden konunun gerçek dünya uygulamalarındaki karşılığı somutlaştırılacaktır.

2. KULLANILAN BAŞLICA FRAMEWORK, KÜTÜPHANE VE ARAÇLAR

JavaScript tabanlı masaüstü uygulama geliştirme ekosistemi, temel olarak web teknolojilerini işletim sistemi katmanına bağlayan çeşitli arayüz araçlarına dayanmaktadır. Bu alanda en yaygın kullanılan ve endüstri standardı haline gelen başlıca araçlar aşağıda detaylandırılmıştır:

2.1 Electron.js

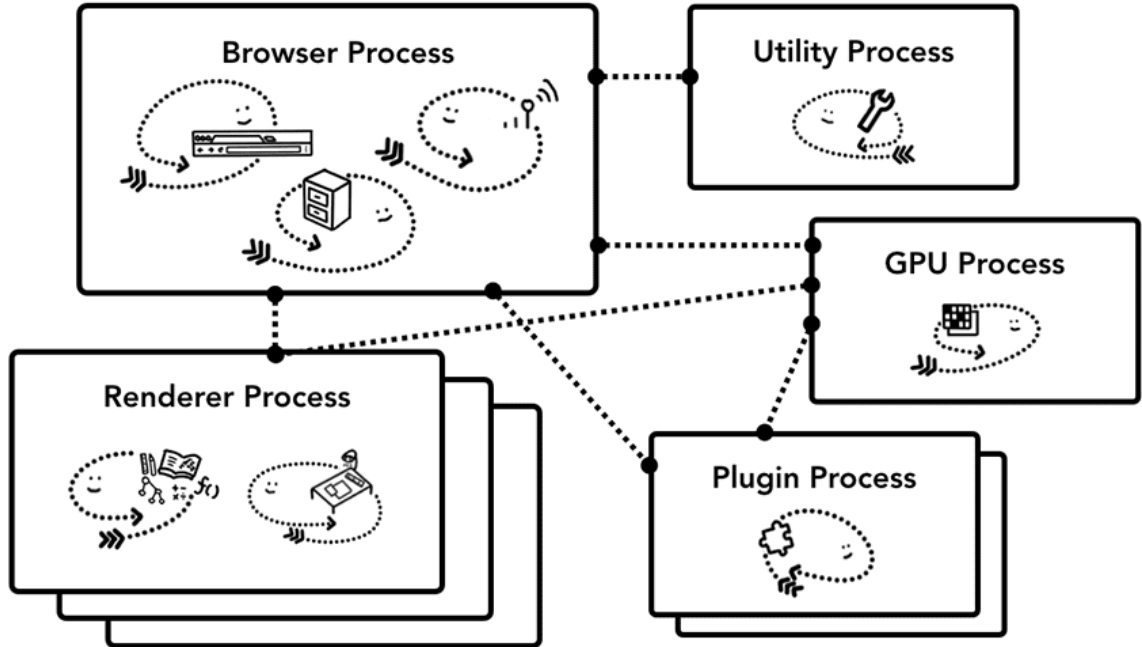
Electron; HTML, CSS ve Javascript kullanarak masaüstü uygulama oluşturmaya yardımcı bir framework olarak tanımlayabiliriz.

Windows, MacOS ve Linux platformları üzerinde uygulamalar oluşturmamıza olanak tanır. Chromium ve Node.js tabanlı çalıştırılabilir bir dosya üzerinden cross-platform uygulama geliştirmemizi ve bunu kullanıcılara dağıtma sürecini hızlandırmamızı sağlar. (Saydam, 2023)

2.1.1 Chromium Motoru

JavaScript tabanlı masaüstü uygulamalarında kullanıcı arayüzü (UI), geleneksel yerel bileşenler yerine web standartları (HTML5 ve CSS3) kullanılarak inşa edilir. Bu arayüzün ekrana yansıtılması ve yönetilmesi görevini **Chromium Motoru** üstlenir.

Chromium'un Mimari yapısını şu şekilde ifade edebiliriz: (bkz. Şekil.1)

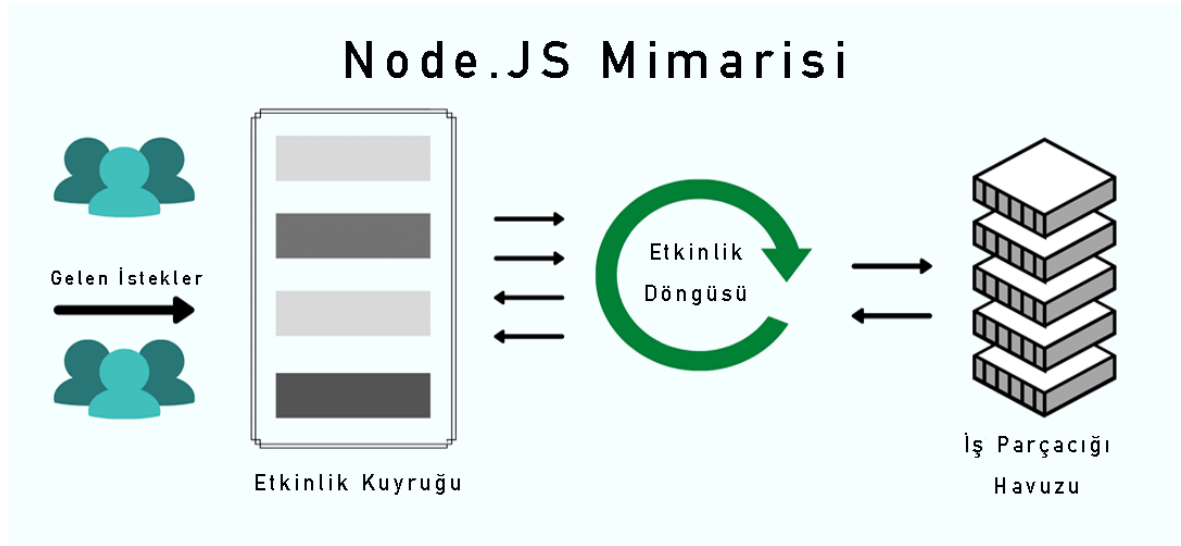


Şekil 1 Chrome'un çok işlemli mimarisinin şeması. Chrome'un her sekme için birden fazla Oluşturucu İşlemi çalıştırdığını göstermek amacıyla Oluşturucu İşlemi altında birden fazla katman gösterilir.

- **Render Süreci:** Chromium, uygulamanın görsel kısmını bir web tarayıcısı gibi işleyerek yüksek grafik performansına sahip, esnek ve dinamik arayüzler sunulmasına olanak tanır.
- **Donanım Hızlandırma:** Modern Chromium motorları, GPU (Grafik İşlemci Birimi) hızlandırma desteği sayesinde karmaşık animasyonların ve yüksek çözünürlüklü içeriklerin akıcı bir şekilde görüntülenmesini sağlar.
- **Geliştirici Araçları (DevTools):** Web geliştiricilerinin alışık olduğu "Insepet Element" (Ögeyi Denetle) gibi araçlar, masaüstü uygulaması geliştirilirken de kullanılabilir. Bu durum, hata ayıklama (debugging) süreçlerini önemli ölçüde hızlandırır.
- **V8 JavaScript Motoru:** Chromium içerisinde yer alan V8 motoru, yazılan JavaScript kodlarını makine diline hızlıca çevirerek uygulamanın tepki süresini (responsiveness) optimize eder.

2.1.2 Node.js

JavaScript normal şartlarda tarayıcı içerisinde "sandbox" (kum havuzu) denilen kısıtlı bir alanda çalışır ve güvenlik gerekçesiyle kullanıcının dosyalarına veya sistem kaynaklarına doğrudan erişemez. Masaüstü uygulama geliştirmede bu kısıtlamayı aşan temel bileşen **Node.js**'tir.



Şekil 2 Node.js JS Mimarisinin temsili gösterimi

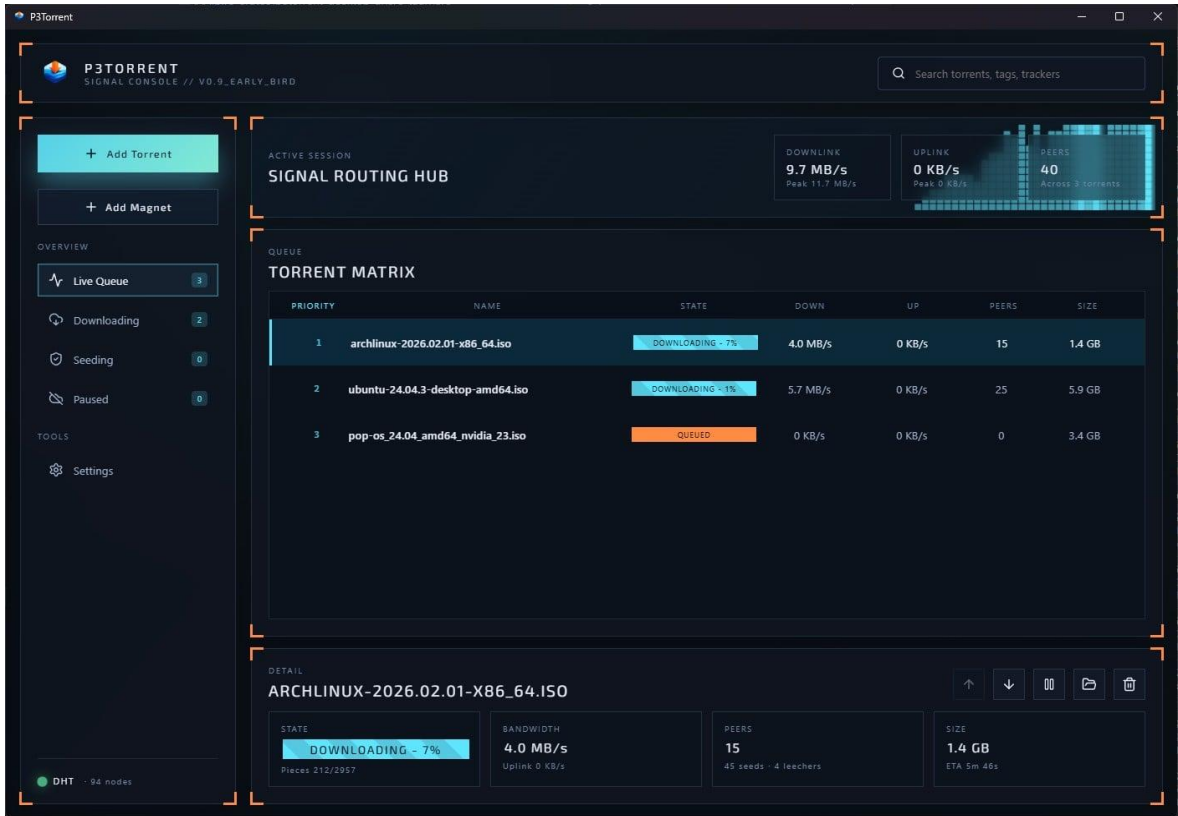
- **Sistem Erişim Yetkinliği:** Node.js, uygulamanın işletim sistemiyle (OS) doğrudan konuşmasını sağlar. Bu sayede uygulama; dosya okuma/yazma, sistem bilgilerini çekme (CPU, RAM kullanımı vb.) ve ağ yapılandırmalarına erişim gibi işlemleri gerçekleştirebilir.
- **V8 Motoru Paylaşımı:** Hem Chromium hem de Node.js, JavaScript kodunu çalıştırmak için Google tarafından geliştirilen V8 motorunu kullanır. Masaüstü uygulamalarında bu iki yapı aynı motor üzerinde uyum içinde çalışarak performans kaybını önler.

- **Olay Döngüsü (Event Loop):** Node.js'in asenkron yapısı sayesinde, uygulama arka planda ağır bir dosya işlemi yaparken (örneğin bir video dosyasını kaydederken) kullanıcı arayüzü donmadan çalışmaya devam eder.
- **Modüler Yapı (NPM):** Node.js entegrasyonu, geliştiricilere milyonlarca hazır kütüphaneyi barındıran NPM (Node Package Manager) ekosistemini masaüstü uygulamalarına taşıma imkânı sunar.

2.2 Tauri

Tauri, Electron'a modern ve performans odaklı bir alternatif olarak geliştirilmiştir. JavaScript ile masaüstü uygulamaları geliştirme prensibini korurken, güvenlik ve kaynak yönetimi konusunda farklı bir yaklaşım sergiler:

- Uygulama geliştirme için güvenli temel
- Sistemin yerel web görünümünü kullanarak daha küçük paket boyutu elde edilir.
- Geliştiricilerin herhangi bir ön uç ve birden fazla dil için bağlayıcılar kullanabilme esnekliği



Şekil 3 Tauri Framework'ü Kullanılarak Geliştirilmiş Modern Bir Masaüstü Uygulaması Arayüzü.

Görsel 3'te görüldüğü üzere, JavaScript tabanlı masaüstü geliştirme araçlarından Tauri, kullanıcılara işletim sisteminin standart ve kısıtlı arayüzlerinin ötesinde, tamamen özelleştirilebilir ve yüksek çözünürlüklü deneyimler sunar. Tauri'nin işletim sisteminin yerel "webview" altyapısını kullanması, bu tarz görsel açıdan zengin ve animasyonlu arayüzlerin çok düşük kaynak tüketimiyle (RAM) çalışmasına imkân tanır. Bu örnek, modern yazılım dünyasında "performans" ve "estetik" dengesinin JavaScript ekosistemiyle nasıl kurulduğunu somutlaştırmaktadır.

Güvenli Temel

Rust üzerine inşa edildiği için Tauri, Rust'ın sunduğu bellek, iş parçacığı ve tür güvenliği avantajlarından yararlanabiliyor. Tauri üzerine inşa edilen uygulamalar, Rust uzmanları tarafından geliştirilmeye gerek kalmadan bu avantajlardan otomatik olarak faydalanabiliyor.

Tauri, büyük ve küçük sürümler için güvenlik denetiminden de geçiyor. Bu denetim sadece Tauri organizasyonundaki kodları değil, Tauri'nin bağımlı olduğu yukarı akış bağımlılıklarını da kapsıyor. Elbette bu tüm riskleri ortadan kaldırmıyor, ancak geliştiricilerin üzerine inşa edebilecekleri sağlam bir temel sağlıyor.

Daha Küçük Uygulama Boyutu

Tauri uygulamaları, her kullanıcının sisteminde zaten mevcut olan web görünümünden yararlanır. Bir Tauri uygulaması yalnızca o uygulamaya özgü kod ve varlıkları içerir ve her uygulamayla birlikte bir tarayıcı motoru paketlemeye gerek duymaz. Bu, minimum bir Tauri uygulamasının 600 KB'den daha küçük olabileceği anlamına gelir.

Esnek Mimari

Tauri web teknolojilerini kullandığı için, neredeyse tüm ön uç çerçeveleri Tauri ile uyumludur. Ön Uç Yapılandırma kılavuzu, popüler ön uç çerçeveleri için yaygın yapılandırmaları içerir.

Invoke JavaScript ve Rust arasındaki bağlantılar, JavaScript'te bu işlevi kullanan geliştiriciler için mevcuttur; Swift ve Kotlin bağlantıları ise Tauri eklentileri için mevcuttur.

TAO, Tauri pencerelerinin oluşturulmasından, WRY ise web görünümünün işlenmesinden sorumludur. Bunlar Tauri tarafından sürdürülen kütüphanelerdir ve Tauri'nin sunduklarının ötesinde daha derin sistem entegrasyonu gerekiyorsa doğrudan kullanılabilirler.

Ek olarak, Tauri, temel Tauri'nin sunduğu özellikleri genişletmek için bir dizi eklenti de sunmaktadır. Bu eklentileri, topluluk tarafından sağlananlarla birlikte Eklentiler bölümünde bulabilirsiniz. (CC-BY/MIT, 2025)

2.3 Diğer Yardımcı Araçlar ve Kütüphaneler

Masaüstü uygulamalarının geliştirilme sürecinde arayüz tasarımı ve veri yönetimi için yardımcı kütüphaneler de kritik rol oynar:

2.3.1. React / Vue.js

Uygulamanın ön yüzündeki bileşen yapısını yönetmek için sıklıkla bu framework'ler entegre edilir.

2.3.2 Native Modules:

JavaScript'in doğrudan erişemediği donanım özelliklerine erişmek için kullanılan yerel kütüphanelerdir.

3. ALANIN AVANTAJLARI VE SINIRLILIKLARI

JavaScript ile masaüstü uygulaması geliştirme yaklaşımı, geleneksel yöntemlere göre belirgin üstünlükler sunsa da beraberinde bazı teknik kısıtlamaları da getirmektedir. Bu avantajlar ve sınırlılıklar aşağıda başlıklar halinde irdelenmiştir.

3.1. Avantajlar

- **Çapraz Platform Desteği:** Tek bir JavaScript kod tabanı kullanılarak Windows, macOS ve Linux işletim sistemleri için uyumlu çıktılar alınabilmektedir. Bu durum, geliştirme maliyetini ve süresini minimize eder.
- **Hızlı Arayüz Geliştirme (UI/UX):** Web ekosistemindeki gelişmiş CSS framework'leri (Bootstrap, Tailwind vb.) sayesinde, geleneksel masaüstü dillerine kıyasla çok daha estetik ve kullanıcı dostu arayüzler kolayca tasarlanabilmektedir.
- **Geniş Kütüphane Desteği:** NPM (Node Package Manager) üzerinden erişilebilen milyonlarca açık kaynaklı kütüphane, uygulama geliştirme sürecinde karmaşık özelliklerin (dosya şifreleme, veri analizi vb.) hızlıca entegre edilmesini sağlar.

3.2. Sınırlılıklar ve Teknik Zorluklar

- **Yüksek Kaynak Tüketimi (RAM ve CPU):** Özellikle Electron.js tabanlı uygulamalar, her pencere için ayrı bir Chromium motoru çalıştırdığından dolayı sistem belleğini (RAM) yoğun bir şekilde kullanır.
- **Büyük Dosya Boyutları:** Uygulama paketinin içerisine tam teşekküllü bir tarayıcı motoru (Chromium) dahil edildiği için, basit bir uygulama bile yüzlerce megabaytlık disk alanı kaplayabilir.
- **Güvenlik Riskleri:** Web tabanlı teknolojilerin kullanımı, uygulamayı XSS (Cross-Site Scripting) gibi web dünyasına özgü saldırılara karşı açık hale getirebilir. Bu nedenle sıkı güvenlik politikalarının uygulanması zorunludur.

3.3. Electron.js ve Tauri Karşılaştırma

Tablo1.Electron.js ve Tauri Framework’lerinin Teknik Karşılaştırması

Özellik	Electron.js	Tauri
Paket Boyutu	Yüksek(100MB+)	Çok Düşük (-10MB)
Bellek (RAM) Kullanımı	Yüksek	Düşük
Güvenlik	Orta	Yüksek(Rust tabanlı)
Öğrenme Eğrisi	Kolay (Sadece JS)	Orta (Rust bilgisi gerekebilir)
Tarayıcı Motoru	Gömülü Chromium	Sistem Webview (Yerel)

Tablo-1 incelendiğinde, modern uygulamaların performans ve güvenlik odaklı olarak Tauri’ye yöneldiği görülmektedir.

4. GERÇEK DÜNYADAN UYGULAMA ÖRNEKLER

JavaScript tabanlı teknolojilerin masaüstü dünyasındaki başarısını kanıtlayan en somut örnekler aşağıda analiz edilmiştir.

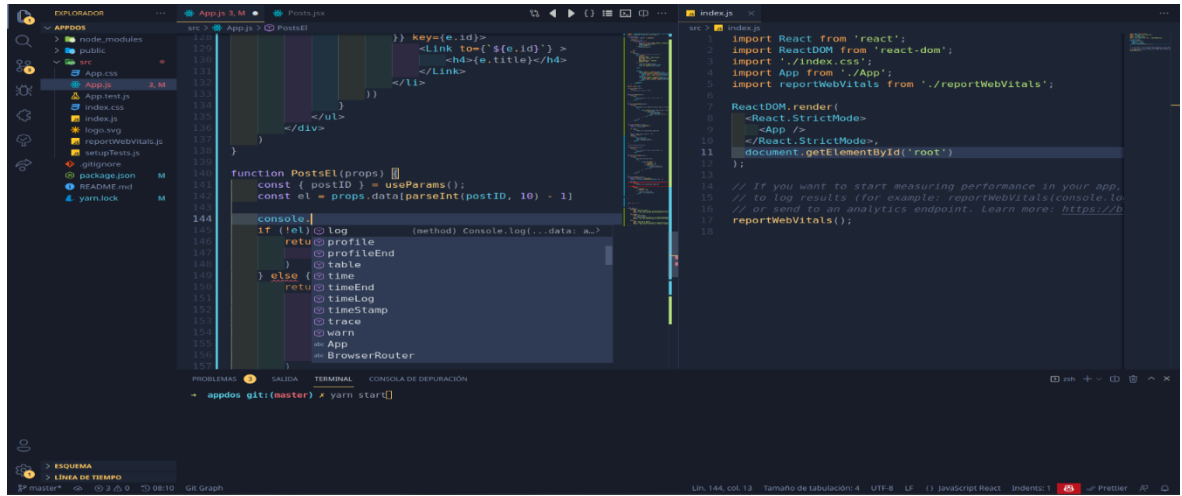
4.1 Visual Studio Code (VS Code)

Visual Studio Code, popüler **kaynak kodu düzenleyicilerden** biridir. Visual Studio Code, açık kaynaklıdır ve kullanımı ücretsizdir. **Windows**, **Linux** ve **macOS** gibi işletim sistemleriyle uyumludur.

Kullanıcı dostu bir arayüzü olan Visual Studio Code, programcıların anlaşılır kodlar yazmasını sağlar. Aynı zamanda programlama sürecini kolaylaştırır. Kullanıcının editörü kullanıma göre değiştirmesine izin verir, bu da kullanıcının kütüphaneleri internetten indirebileceği ve gereksinimlerine göre kodla entegre edebileceği anlamına gelir.

4.1.1 VS Code'un Avantajları

1. Platformlar arası destek sunar.
2. Hafiftir.
3. Açık kaynaklı ve ücretsizdir. Herhangi bir lisans ödmeden kullanılabilir.
4. HTML, CSS, JSON gibi web teknolojileri için de destek sağlar.
5. Visual Studio Code, kullanımı kolay bir arayüze sahiptir.
6. Kod yazma işlemini daha hızlı ve verimli hale getirir.
7. Yüksek performanslı sunar.
8. Visual Studio Code, sürekli güncellemeler alır. Bu güncellemeler, yeni özellikler, hata düzeltmeleri ve performans iyileştirmeleri gibi öğeler içerir. (Coderspace, 2024)



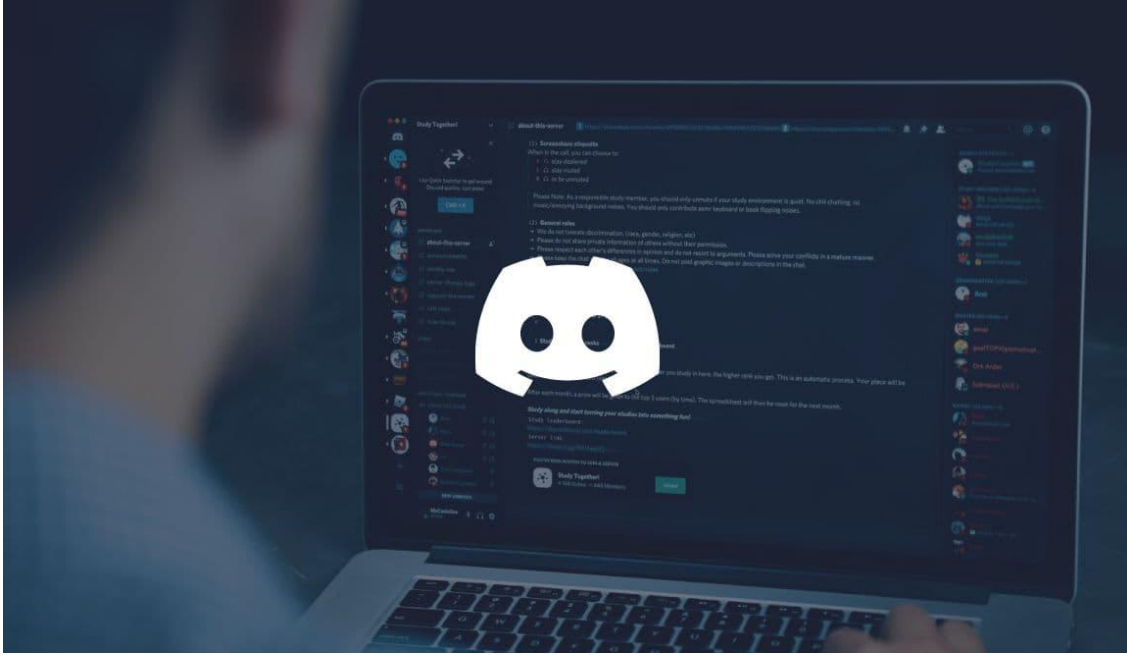
Şekil 4 Visual Studio Code Arayüzünde JavaScript Geliştirme Süreci ve IntelliSense Özelliği.

Görsel Analizi ve Teknik Değerlendirme: Görsel 5'te, Electron.js framework'ü ile geliştirilmiş olan Visual Studio Code editörünün çalışma arayüzü görülmektedir. Bu örnek, JavaScript'in masaüstü dünyasındaki gücünü şu teknik detaylarla kanıtlamaktadır:

- **IntelliSense ve Otomatik Tamamlama:** Görselin orta kısmında görülen console. Komutu sonrası açılan menü, editörün arka planda çalışan Node.js altyapısı sayesinde kodu gerçek zamanlı analiz ettiğini gösterir.
- **Modern Arayüz Esnekliği:** HTML/CSS tabanlı bir arayüzün, geleneksel masaüstü uygulamalarına kıyasla ne kadar modern, özelleştirilebilir ve geliştirici dostu bir yapı sunabileceği bu görsel üzerinden net bir şekilde anlaşılmaktadır.

4.2. Discord

Discord; her çeşitten kullanıcının kullandığı uygulamalar arasında olmakla beraber daha çok oyun kullanıcılarının tercih ederek kullandığı, arkadaş topluluklarıyla ya da oyun topluluklarıyla ve oyun geliştiricileriyle hem sesli hem yazılı hem de görüntülü görüşmeyi ve sohbet etmeyi sağlayan ve tüm bunların ücretsiz olarak yapıldığı yani ücretsiz olarak kullanılabilen bir platform olarak biliniyor. (Techcareer, 2023)



Şekil 5 Discord Masaüstü Uygulaması: Web Teknolojileri ve Yerel Modüllerin Entegrasyonu

- **Çapraz Platform Tutarlılığı:** Discord; web, masaüstü (Windows, macOS, Linux) ve mobil sürümlerinde benzer bir kullanıcı deneyimi sunar. Bu tutarlılık, Electron.js sayesinde web teknolojilerinin masaüstüne doğrudan taşınmasıyla sağlanmıştır.
- **Hibrit Performans Yaklaşımı:** Discord mühendisleri, uygulamanın arayüzünü JavaScript/React ile geliştirirken, sesli iletişim gibi yüksek performans gerektiren "düşük seviyeli" işlemler için C++ ile yazılmış yerel modülleri JavaScript'e bağlamıştır. Bu, "Alan sınırlılıklarını" aşmak için kullanılan çok yaratıcı bir çözümdür.
- **Hızlı Güncelleme Mekanizması:** Uygulama, içeriğinin büyük bir kısmını web üzerinden yüklediği için kullanıcıların her küçük değişiklikte tüm uygulamayı yeniden indirmesine gerek kalmaz.

VS Code, Electron framework'ünü yüksek performanslı bir metin işleme aracı olarak kullanırken; Discord, aynı altyapıyı karmaşık ağ operasyonları ve gerçek zamanlı iletişim süreçlerini yönetmek için modernize etmiştir. Bu iki örnek, JavaScript'in masaüstü dünyasında ne kadar farklı ihtiyaçlara cevap verebildiğini kanıtlamaktadır.

5. BASİT BİR KOD PARÇASI (SNIPPET) İLE ÖRNEK GÖSTERİM

JavaScript tabanlı bir masaüstü uygulaması geliştirmek için öncelikle ana sürecin yapılandırılması gerekir. Aşağıdaki kod bloğu, Electron.js framework'ü kullanılarak boş bir uygulama penceresi oluşturmanın en temel yolunu göstermektedir:

```
// Electron modüllerini içe aktarma

const { app, BrowserWindow } = require('electron');

function createWindow() {

  // Yeni bir tarayıcı penceresi oluşturma

  const win = new BrowserWindow({

    width: 800,      // Pencere genişliği

    height: 600,     // Pencere yüksekliği

    webPreferences: {

      nodeIntegration: true // Node.js özelliklerini kullanıma açma

    }

  });

  // Pencere içerisinde görüntülenecek HTML dosyasını yükleme

  win.loadFile('index.html');

}

// Uygulama hazır olduğunda pencereyi oluştur

app.whenReady().then(createWindow);
```

Şekil 7 Bu kod örneği Yapay Zekâdan alınmıştır.

Kodun Teknik Analizi:

- **BrowserWindow:** Chromium motorunu kullanarak işletim sistemi üzerinde yeni bir pencere nesnesi üretir.
- **LoadFile:** Uygulamanın arayüzünü oluşturan standart bir web dosyasını (HTML) masaüstü penceresine yansıtır.
- **app.whenReady:** İşletim sisteminin ve Electron çekirdeğinin uygulama başlatılmaya uygun hale gelmesini bekleyen bir olay yöneticisidir (event listener).

6. SONUÇ VE DEĞERLENDİRME

Rapor kapsamında, JavaScript programlama dili yalnızca web tarayıcılarıyla sınırlı kalmamış masaüstü uygulama geliştirme süreçlerinde nasıl kilit bir rol oynadığı teknik detaylarıyla incelenmiştir. Özellikle Electron.js ve Tauri gibi framework'lerin üzerine yapılan araştırma, web teknolojilerinin (HTML, CSS ve JavaScript) masaüstü platformlarının sunduğu sistem kaynaklarıyla birleşiminin yazılım dünyasına kazandırdığı esnekliği açıkça ortaya koymaktadır.

Sektörün önde gelen uygulamalarından olan Visual Studio Code ve Discord üzerine detaylı bir araştırılma yapılmış ve JavaScript tabanlı bir mimarinin hem karmaşık veri yönetimini hem de kullanıcı dostu arayüzleri aynı anda sunabileceğini kanıtlanmıştır. Bununla birlikte, yapılan performans analizleri sonucu bu teknolojilerin sistem kaynaklarını geleneksel yerel uygulamalara oranla daha yoğun kullandığını görmekteyiz. Geliştiriciler projenin gereksinimlerine göre Electron.js'in sunduğu geniş kütüphane desteği ile Tauri'nin sunduğu yüksek performans ve güvenlik odaklı yapı arasında stratejik bir tercih yapmalarını zorunlu kıldığını görmekteyiz.

Kaynakça

CC-BY/MIT. (2025, Ağustos 1). Tauri: https://v2-tauri-app.translate.google.com/start/?_x_tr_sl=en&_x_tr_tl=tr&_x_tr_hl=tr&_x_tr_pto=tc adresinden alındı

Coderspace. (2024, Eylül 5). *Coderspace*. Coderspace: <https://coderspace.io/blog/visual-studio-code-nedir-ne-ise-yarar/> adresinden alındı

Saydam, E. (2023, Kasım 11). *Medium*. Electron: Javascript, HTML ve CSS ile platformlar arası masaüstü uygulamalar oluşturma: <https://blog.ekremsaydam.com.tr/electron-javascript-html-ve-css-ile-platformlar-aras%C4%B1-masa%C3%BCst%C3%BC-uygulamalar-olu%C5%9Fturma-62a9db8e9a2b> adresinden alındı

Techcareer. (2023, Temmuz 24). *Techcareer.net*. Techcareer.net: <https://www.techcareer.net/blog/discord-nedir> adresinden alındı